

git

parte 1: trabajando en local

¿Qué es un sistema de control de versiones?

Un **Sistema de Control de Versiones (SCV)** es una aplicación que permite gestionar los cambios que se realizan en un proyecto, guardando así versiones del mismo en todas sus fases de desarrollo.

¿Qué aporta un SCV?

- Registra cada cambio en el proyecto, quién y cuándo lo hace.
- Permite volver a estados previos del desarrollo.
- Permite gestionar diferentes versiones del proyecto para trabajar en paralelo y luego fusionarlas.
- Permite colaborar entre diferentes usuarios, facilitando la resolución de conflictos.

¿Qué SCV existen?

git es, hoy en día, el SCV más usado con diferencia. Otras alternativas:

- Mercurial
- Subversion (SVN)
- Perforce (*muy usado en desarrollo de videojuegos*)

Git LFS **no** es una alternativa a git: es un plugin que se instala sobre git para gestionar ficheros binarios muy pesados (vídeos, assets...).

¿Por qué usar git?

- Creado en 2005.
- El SCV más extendido.
- Arquitectura distribuida.
- De código abierto.
- Manejo eficiente de ramas.

Comandos que veremos hoy

- `git config`
- `git init`
- `git status`
- `git add`
- `git commit`
- `git log`

Y además: el fichero `.gitignore` y, si hay tiempo, `git diff`.

git config

```
git config [opciones] [clave] [valor]
```

El comando `git config` se utiliza para establecer y configurar las opciones de Git:

- Nombre de usuario
- Correo electrónico
- Varias preferencias que controlan el comportamiento de Git

Es esencial al principio, ya que Git necesita saber quién eres (nombre y correo electrónico) para registrar adecuadamente tus cambios.

git config — ejemplo de uso

Configuración global — afecta a todos los repositorios

```
git config --global user.name "nombre"  
git config --global user.email "email"
```

Configuración local — afecta al repositorio donde estás trabajando

```
git config user.name "nombre"  
git config user.email "email"
```

git config — comprobar la configuración

Si algo falla, primero comprueba qué tienes configurado:

```
git config --list  
git config --global --list  
  
git config user.name  
> nombre
```

Si el nombre o el email salen vacíos, git no podrá asociarte los commits correctamente.

git init

```
git init
```

Su función principal es crear un **nuevo repositorio de Git** en un directorio, permitiendo que el proyecto en ese directorio sea gestionado por Git.

¿Qué hace `git init`?

1. **Crea un nuevo repositorio de Git:** ejecutar `git init` en un directorio (vacío o con un proyecto existente) lo inicializa como repositorio de Git.
2. **Directorio oculto `.git`:** se crea una carpeta oculta `.git` donde Git guarda toda la información necesaria para gestionar el historial (commits, ramas, etc.).
3. **No afecta los archivos existentes:** si el directorio ya tiene archivos, `git init` no los modifica; simplemente prepara el directorio para comenzar a rastrear cambios.

git init — ejemplo

Crear un nuevo repositorio desde cero:

```
mkdir mi-proyecto
cd mi-proyecto
git init
> Initialized empty Git repository in /ruta/a/mi-proyecto/.git/

ls -lart mi-proyecto
> total 7096
drwxr-xr-x  38 jose  staff   1216 23  sep 10:27 ..
drwxr-xr-x  12 jose  staff    384 23  sep 10:27 .git
```

Los 3 estados de un fichero

Cuando trabajas en local, cada fichero de tu proyecto está en uno de estos 3 estados:

1. **Untracked** — git no lo conoce todavía (fichero nuevo).
2. **Modified** — git lo conoce, pero tiene cambios sin preparar.
3. **Staged** — está preparado, listo para el próximo commit.

Los comandos que vienen ahora (`status` , `add` , `commit`) existen precisamente para movernos entre estos 3 estados.

3 comandos en grupo

- `git status`
- `git add`
- `git commit`

git status

El comando `git status` proporciona información sobre el estado actual de tu repositorio:

- Qué archivos han cambiado
- Cuáles están listos para ser confirmados (committed)
- Cuáles no están siendo rastreados (untracked)

Este comando **no modifica el estado de los archivos ni realiza cambios en el repositorio**, solo te informa sobre su estado actual.

git status — ejemplo

```
git status
```

```
On branch main
```

```
No commits yet
```

```
Untracked files:
```

```
(use "git add <file>..." to include in what will be committed)
```

```
    README.md
```

```
    hola.txt
```

```
nothing added to commit but untracked files present (use "git add" to track)
```

Fíjate: git ya te sugiere el siguiente comando (`git add`).

git add

```
git add <archivo>    # añade un fichero concreto
git add .             # añade todos los cambios del directorio actual
```

El comando `git add` se utiliza para agregar archivos al **área de preparación** (staging area), un paso intermedio antes de confirmar los cambios en el repositorio con un commit.

Es esencial para decirle a Git qué cambios (nuevos archivos, archivos modificados, etc.) deben incluirse en el próximo commit.

¿Qué hace `git add`?

1. **Prepara los archivos para el commit:** `git add` toma los archivos o cambios y los coloca en el área de preparación. Esto no los guarda de forma permanente en el historial; solo los prepara para el commit.
2. **No realiza un commit:** no guarda los cambios de manera definitiva en el historial del proyecto, solo los marca para que sean incluidos en el próximo commit.
3. **Agrega archivos nuevos y cambios en archivos existentes:** sirve para archivos nunca rastreados por Git, así como para actualizar archivos modificados que ya están siendo rastreados.
4. **Cuidado con `git add .`:** añade *todo* lo que haya en el directorio. Si no tienes un `.gitignore`, puedes acabar versionando ficheros que no querías.

git commit

```
git commit -m "Mensaje descriptivo del commit"
```

Uno de los comandos más importantes en Git, ya que se utiliza para **guardar los cambios en el historial** del repositorio.

Después de agregar archivos al área de preparación con `git add`, el siguiente paso es confirmarlos usando `git commit`. Esto crea un "snapshot" o instantánea de los cambios en tu proyecto, permitiendo revertir a este punto en el futuro si es necesario.

¿Qué hace `git commit`?

1. **Confirma los cambios en el área de preparación:** `git commit` toma los archivos que has preparado con `git add` y los guarda en el historial del repositorio, creando un "punto de control".
2. **Crea un mensaje de commit:** cada commit debe ir acompañado de un mensaje descriptivo que explique qué cambios se han realizado.
3. **No afecta los archivos que no han sido preparados:** solo los archivos en el área de preparación (staging area) se incluyen en el commit.

`git commit` — cuidado con esto

Si ejecutas `git commit` sin `-m`, git abrirá un editor de texto (normalmente Vim) para que escribas el mensaje.

Si te quedas "atrapado" en Vim:

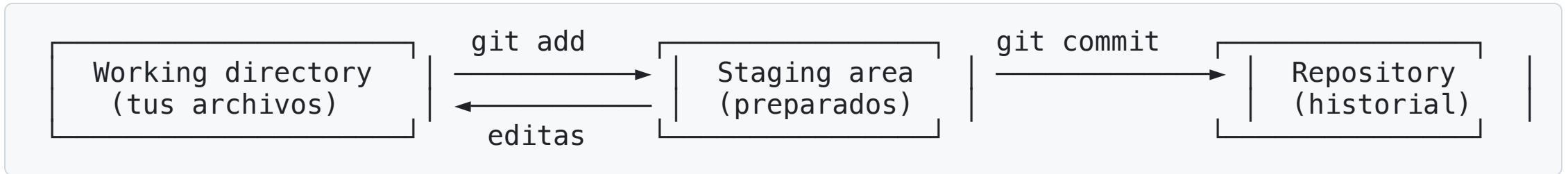
1. Pulsa `Esc`
2. Escribe `:wq` y pulsa `Enter`

O, más sencillo: usa siempre `git commit -m "mensaje"`.

Ejemplo completo

```
mkdir mi-proyecto  
cd mi-proyecto  
git init  
touch README.md  
touch hola.txt  
git status  
git add README.md  
git status  
git commit -m "Confirmo cambios en el fichero README.md"  
git status
```

El flujo de trabajo de Git



Es un **ciclo**: después de un commit sigues editando en el working directory, y vuelves a empezar.

.gitignore

No todo lo que hay en tu carpeta de proyecto debe ir a git: ficheros temporales, carpetas de dependencias, credenciales, archivos generados...

El fichero `.gitignore` le dice a git qué ficheros y carpetas **ignorar** (no aparecerán nunca en `git status` ni se podrán añadir con `git add`).

Se crea como un fichero de texto normal en la raíz del proyecto.

`.gitignore` — ejemplo

```
# ficheros de log
*.log

# carpeta de dependencias
node_modules/

# ficheros de configuración local con datos sensibles
.env

# ficheros del sistema operativo / editor
.DS_Store
.vscode/
```

Buena práctica: crea el `.gitignore` **antes** de tu primer `git add .`, así evitas versionar por error algo que no quieres.

git log

Es una herramienta fundamental en Git que te permite ver el historial de commits de un repositorio. Muestra una lista de todos los commits que se han realizado, permitiendo explorar quién hizo los cambios, cuándo se hicieron, qué mensajes de commit se usaron y más.

¿Qué hace `git log`?

Muestra información detallada sobre cada commit en el historial del repositorio:

1. **SHA-1 hash:** el identificador único del commit (una larga cadena hexadecimal).
2. **Autor:** la persona que hizo el commit.
3. **Fecha:** el momento en que se realizó el commit.
4. **Mensaje del commit:** el mensaje asociado que describe los cambios realizados.

git log — ejemplo

```
commit 3f1e948f8721d28f1843f2f (HEAD -> main)
Author: Juan <juan@exale.com>
Date: Tue Sep 14 15:20:42 2024 +0200
```

Actualización de los estilos CSS y correcciones menores en el HTML

```
commit 124c35fe50cfbb5a3c44b8f74d8c4df8799d5df0
Author: María García <maria@example.com>
Date: Mon Sep 13 10:34:08 2024 +0200
```

Implementación de la nueva funcionalidad de autenticación

git log --oneline

Para un historial más compacto, muy útil cuando ya llevas muchos commits:

```
git log --oneline
```

```
3f1e948 Actualización de los estilos CSS y correcciones menores en el HTML  
124c35f Implementación de la nueva funcionalidad de autenticación
```

Cada línea: hash corto + mensaje del commit.

`git diff` *(extra, si hay tiempo)*

Antes de hacer `git add`, puedes ver **exactamente qué has cambiado** línea a línea:

```
git diff
```

```
diff --git a/README.md b/README.md
--- a/README.md
+++ b/README.md
-# Mi proyecto
+# Mi proyecto de prácticas
```

– líneas eliminadas, + líneas añadidas.

Buenas prácticas de commits

- Mensajes en **imperativo**: "Añade validación de formulario", no "Añadido" ni "Añadiendo".
- Un commit = **un cambio lógico**. Evita commits gigantes que mezclan varias cosas.
- Commitea a menudo: es más fácil volver atrás si algo falla.
- Nunca comitees credenciales, contraseñas o claves (para eso está `.gitignore`).

Resumen — chuleta de comandos

Comando	Para qué sirve
<code>git config --global user.name/email</code>	Identificarte ante git
<code>git init</code>	Crear un repositorio nuevo
<code>git status</code>	Ver el estado de tus ficheros
<code>git add <fichero> / git add .</code>	Preparar cambios para el commit
<code>git commit -m "..."</code>	Guardar los cambios en el historial
<code>git log / git log --oneline</code>	Ver el historial de commits
<code>.gitignore</code>	Excluir ficheros del control de versiones
<code>git diff</code>	Ver los cambios línea a línea

¿Dudas?

Próxima sesión: ramas y trabajo en remoto (GitHub)